Operating Systems and Parallel Computing

Lecture 9

Prof. Matteo Turilli

From RAM to Storage

- Applications need to store and retrieve information.
- RAM alone does not meet every storage requirement:
  a. A dataset may be larger than the available RAM.
  b. Process memory is normally released when a process ends, and RAM loses its contents when power is removed.
  c. Processes have separate address spaces, private to each process.
- Long-term storage therefore needs to provide:
  a. Capacity for large amounts of information.
  b. Persistence beyond the lifetime of a process or a power cycle.
  c. Controlled access to shared information by multiple processes.
- Examples of storage media:
  a. Hard disk drives (HDDs) store data on magnetic platters.
  b. Solid-state drives (SSDs) have no moving parts and lower access latency.
  c. Magnetic tape is also used for backups and archives.









Challenges and a New Abstraction

- Shared storage raises three questions:
  a. How do we find information?
  b. How do we control who can access it?
  c. How do we track which parts of the storage are free?
- The **file** abstraction gives programs a logical unit for storing information.
- A **process** executes instructions, an **address space** holds its memory, and a **file** can retain its inputs and outputs.
- Processes create files and read or update existing ones through OS interfaces.
- Files on persistent storage can survive a process ending or a system restart. Buffered writes must reach storage to survive power loss.
- The **file system** organizes files, names and metadata, and works with the OS to manage storage and enforce access rules.

Studying the File System

- From a user's perspective, we will study:
  a. File names, types and encodings explain how programs identify and interpret stored bytes.
  b. Inodes, directories, paths and links explain how the OS locates files and their metadata.
  c. Users, groups, UID/GID and permission bits explain access rights. We interpret `ls -l` and introduce `sudo`.
- File operations connect storage to processes. We examine sequential and random access, and how an executable becomes part of a process.
- From an implementer's perspective, we will study:
  a. Partitions and UEFI explain where file systems reside and how the system begins loading an OS.
  b. FAT, indexes and extents explain how the file system locates the blocks that hold a file's data.
  c. Block size, caching, compression and deduplication affect performance and the space needed to store data.
  d. Free-space tracking, quotas, journaling, consistency checks and backups support storage management and recovery.

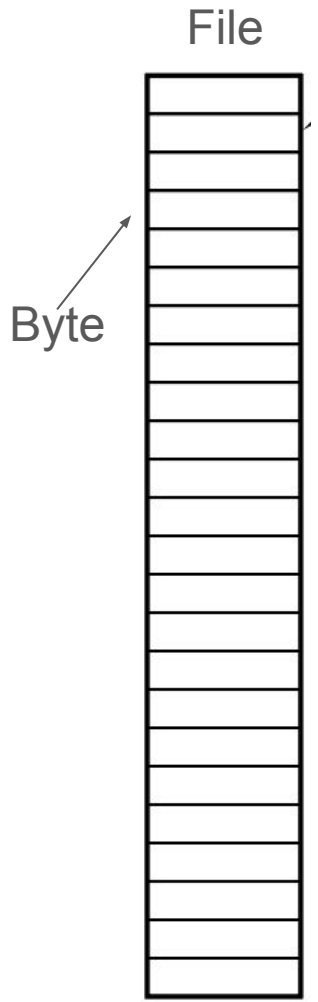
File Naming

- File-name rules depend on the OS and file system.
- Limits apply to each name and to the whole path.
- E.g., On a Linux filesystem (e.g., ext4), a name can contain up to 255 bytes.
- Case handling also varies:
  a. Unix file systems are commonly case sensitive, so `Data.csv` and `data.csv` can name different files.
  b. Windows is usually case insensitive, but case-sensitive directories are possible.
- An extension is a naming convention, e.g., `.csv` or `.py`.
  a. Applications and desktop tools may use extensions to choose how to open files.
  b. Some applications inspect the file content or other metadata to choose how to open the files, independent on their extensions.
  c. Changing an extension does not convert the contents. A CSV renamed `.xlsx` is still CSV data.



File Structure

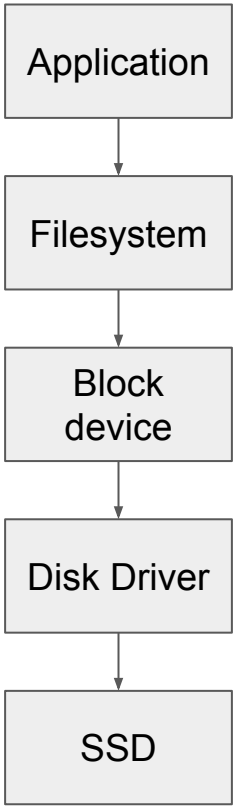
- A regular file is presented to a program as a sequence of bytes.
- Text is stored as bytes using an **encoding**, such as UTF-8. Different characters can require different numbers of bytes (e.g., Kanji).
- A **file format** defines how the contents are organised
- The OS provides access to those bytes. The application decodes them and interprets them according to the file's format (e.g.,CSV)
- A program can read sequentially or seek to a particular byte position, called a byte offset, so it need not always read from the beginning.




```
analyst@ospc:~/data$ head -2 sales_2025_sample.csv | od -Ax -t x1z -v
000000 6f 72 64 65 72 5f 69 64 2c 6f 72 64 65 72 5f 64  >order_id,order_d<
000010 61 74 65 2c 72 65 67 69 6f 6e 2c 70 72 6f 64 75  >ate,region,produ<
000020 63 74 5f 63 61 74 65 67 6f 72 79 2c 70 72 6f 64  >ct_category,prod<
000030 75 63 74 5f 69 64 2c 71 75 61 6e 74 69 74 79 2c  >uct_id,quantity,<
000040 75 6e 69 74 5f 70 72 69 63 65 2c 63 75 73 74 6f  >unit_price,custo<
000050 6d 65 72 5f 69 64 2c 63 68 61 6e 6e 65 6c 0d 0a  >mer_id,channel..<
000060 31 2c 32 30 32 35 2d 31 31 2d 32 34 2c 49 62 65  >1,2025-11-24,Ibe<
000070 72 69 61 2c 53 70 6f 72 74 73 2c 50 35 35 30 36  >ria,Sports,P5506<
000080 2c 38 2c 35 32 2e 30 37 2c 43 32 33 34 33 34 2c  >,8,52.07,C23434,<
000090 73 74 6f 72 65 0d 0a                             >store..<
000097
```

File Types

- **Regular files** contain data such as a CSV table or a program.
- **Directories** associate names with files and other directories.
- **Device files** expose interfaces to devices through the OS:
  - Reads and writes are handled by a device driver, so they need not behave like reads and writes to a regular file.
- **Character devices** provide a byte-oriented interface, e.g., a terminal:
  - Character devices let programs read or write streams of bytes. For example, a terminal delivers keyboard input to a program and displays the output it receives.
- **Block devices** provide access to addressable storage blocks, e.g., on an SSD:
  - An on-disk file system can organize its files and metadata on a block device, as shown on the diagram.



Text Files Encoding

- ASCII is a minimal encoding, while UTF-8 can represent a much larger set of characters.
- ASCII assigns values 0–127 to letters, digits, punctuation and control characters such as a newline.
- An editor must decode those bytes to display text. The wrong encoding can produce unreadable characters.
- Text editors such as VS Code, vi and Notepad can edit plain-text files.
- Text makes shell pipelines convenient, but programs must still agree on the encoding and data format.

Dec	Hex	Chr
0	00	NUL
1	01	SOH
2	02	STX
3	03	ETX
4	04	EOT
5	05	ENQ
6	06	ACK
7	07	BEL
8	08	BS
9	09	HT
10	0A	LF
11	0B	VT
12	0C	FF
13	0D	CR
14	0E	SO
15	0F	SI
16	10	DLE
17	11	DC1
18	12	DC2
19	13	DC3
20	14	DC4
21	15	NAK
22	16	SYN
23	17	ETB
24	18	CAN
25	19	EM
26	1A	SUB
27	1B	ESC
28	1C	FS
29	1D	GS
30	1E	RS
31	1F	US

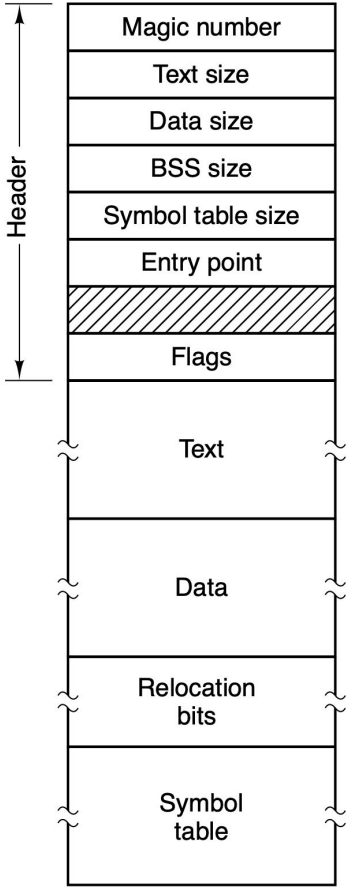
Dec	Hex	Chr
32	20	Space
33	21	!
34	22	"
35	23	#
36	24	$
37	25	%
38	26	&
39	27	'
40	28	(
41	29	)
42	2A	*
43	2B	+
44	2C	,
45	2D	-
46	2E	.
47	2F	/
48	30	0
49	31	1
50	32	2
51	33	3
52	34	4
53	35	5
54	36	6
55	37	7
56	38	8
57	39	9
58	3A	:
59	3B	;
60	3C	<
61	3D	=
62	3E	>
63	3F	?

Dec	Hex	Chr
64	40	@
65	41	A
66	42	B
67	43	C
68	44	D
69	45	E
70	46	F
71	47	G
72	48	H
73	49	I
74	4A	J
75	4B	K
76	4C	L
77	4D	M
78	4E	N
79	4F	O
80	50	P
81	51	Q
82	52	R
83	53	S
84	54	T
85	55	U
86	56	V
87	57	W
88	58	X
89	59	Y
90	5A	Z
91	5B	[
92	5C	\
93	5D	]
94	5E	^
95	5F	_

Dec	Hex	Chr
96	60	`
97	61	a
98	62	b
99	63	c
100	64	d
101	65	e
102	66	f
103	67	g
104	68	h
105	69	i
106	6A	j
107	6B	k
108	6C	l
109	6D	m
110	6E	n
111	6F	o
112	70	p
113	71	q
114	72	r
115	73	s
116	74	t
117	75	u
118	76	v
119	77	w
120	78	x
121	79	y
122	7A	z
123	7B	{
124	7C	|
125	7D	}
126	7E	~
127	7F	DEL

Binary File Formats

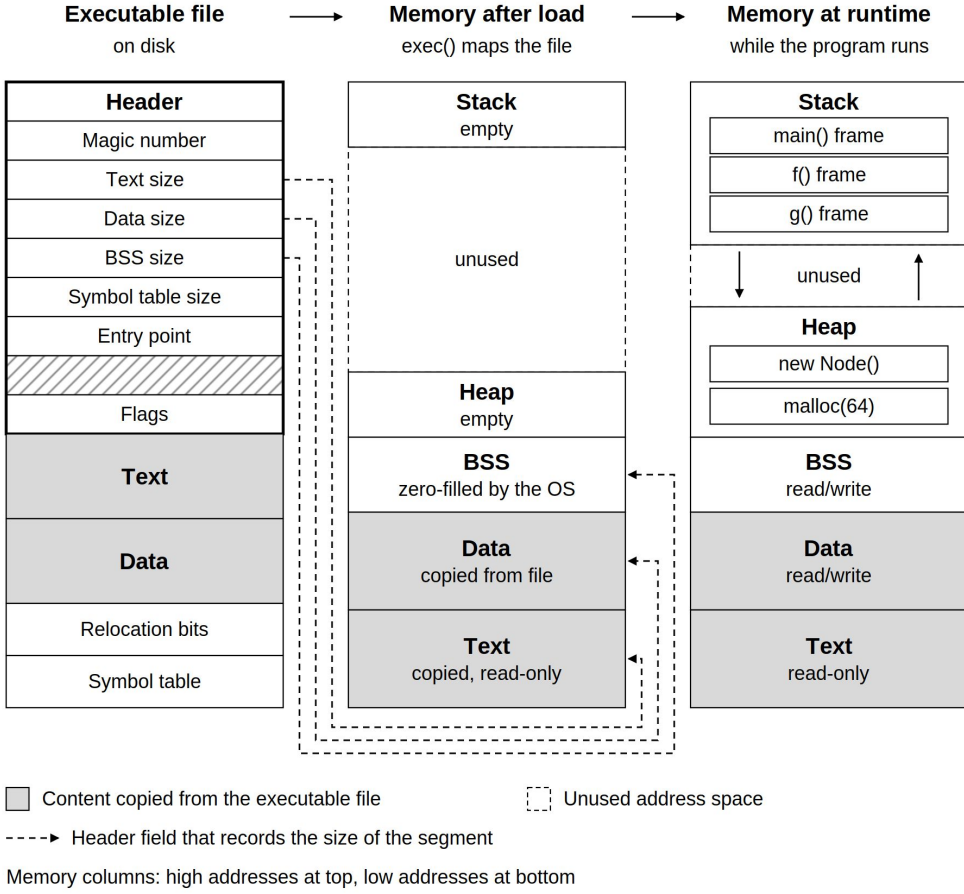
- A **binary format** gives bytes a structure that a particular program understands. E.g., `.docx`, `.pdf`, `.exe`.
- A binary file's format defines how to interpret its different parts.
- Some parts may represent numbers or other data, while others contain text with a specified character encoding.
- There is no single character encoding for the whole file.
- A program needs the right format reader. Renaming a file does not make its bytes valid in another format.
- The figure shows a historical UNIX executable format:
  - The header identifies the format and describes sizes and the entry point.
  - Text contains instructions, while Data contains initialized program data.
  - Relocation information supports linking. Symbols can help with debugging.
  - Modern Linux commonly uses ELF. Execution also requires suitable permissions and loader support.



Historical UNIX executable format

Binary Executable Files: From Disk to Process

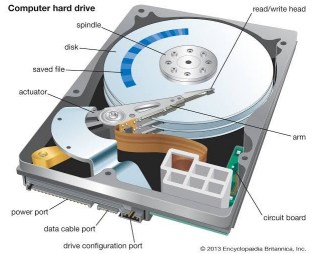
- From a file to an address space
- When a program is loaded:
  a. The OS maps executable segments into virtual memory (i.e., not resident)
  b. Text holds instructions, while Data holds initialized variables.
  c. BSS holds uninitialized variables, i.e., without known value at start time.
  d. The OS and runtime set up the stack and start execution.
- While the program runs:
  a. The CPU reads instructions and accesses program data.
  b. Stack frames hold call state and are released on return.
  c. Heap data can survive a function returning.

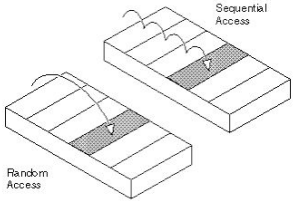


File Access

- **Sequential access** reads bytes in order from the current position:
  - A program may stop before the end. To reread a regular file, it can return to an earlier position.
  - This access pattern suits media like magnetic tape.
- **Random access** lets a program seek to a byte offset before reading or writing:
  - Regular files normally support both sequential and random access.
  - Seeking changes the offset for an open file. It does not itself read data or physically move a disk head.
- Example: an indexed reservation file can locate one record without scanning all earlier records.
- The storage device affects performance and trade offs:
  - An HDD may need to move its head, so scattered access can be slow.
  - An SSD has no moving head, but access size and locality still matter.







File Attributes

- An **inode** holds a file's **metadata**, e.g., size and permissions.
- Each inode has a number unique to its file system.
- The directory stores the name and inode number.
- The table combines attributes from several operating systems.
- E.g.,
  a. Linux `change time` records inode changes, not creation.
  b. `modification time` tracks changes to the contents.

Attribute	Meaning
Protection	Who can access the file and in what way
Password	Password needed to access the file
Creator	ID of the person who created the file
Owner	Current owner
Read-only flag	0 for read/write; 1 for read only
Hidden flag	0 for normal; 1 for do not display in listings
System flag	0 for normal files; 1 for system file
Archive flag	0 for has been backed up; 1 for needs to be backed up
ASCII/binary flag	0 for ASCII file; 1 for binary file
Random access flag	0 for sequential access only; 1 for random access
Temporary flag	0 for normal; 1 for delete file on process exit
Lock flags	0 for unlocked; nonzero for locked
Record length	Number of bytes in a record
Key position	Offset of the key within each record
Key length	Number of bytes in the key field
Creation time	Date and time the file was created
Time of last access	Date and time the file was last accessed
Time of last change	Date and time the file was last changed
Current size	Number of bytes in the file
Maximum size	Number of bytes the file may grow to

File Operations

Common file operations and Linux interfaces

Operation	Description
Create	Creates an empty file with metadata. Linux `open()` with `O_CREAT` can create it.
Delete	Removes a name with `unlink()`. Space can be reclaimed when no hard links or open references remain.
Open	Checks access and returns a file descriptor for later I/O. It does not read the whole file.
Close	Releases the file descriptor. Closing alone does not guarantee that writes have reached persistent storage. See p. 25.
Read	Copies up to the requested bytes into a buffer, normally advancing the offset.
Write	Writes at the offset, or at the end in append mode. Check how many bytes were written.

File Operations

Common file operations and Linux interfaces

Operation	Description
Append	Writes at the end of the file. Linux uses `write()` on a file opened with `O_APPEND`, rather than a separate append syscall.
Seek	Changes the offset used by later reads or writes on a seekable file. It does not itself transfer data. Linux provides `lseek()`.
Get attributes	Retrieves metadata such as size, ownership and permissions, e.g., through `stat()`.
Set attributes	Changes permitted metadata fields, e.g., `chmod()` changes permission bits. The process must have authority to make the change.
Rename	Changes a name or location within the same file system without copying the file's contents. Across file systems, `mv` normally copies the data and then removes the source.

Directories

- **Directories** are a type of file that organizes names.
- In Linux, each directory entry links a name to an inode number.

Directory ~/data contains an entry:
inode: 1925551
name: sales_2025_sample.csv



Directories

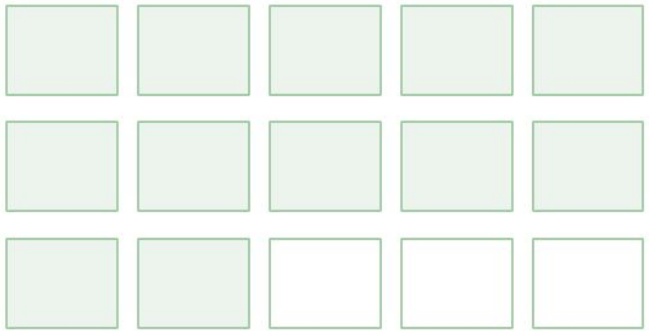
- **Directories** are a type of file that organizes names.
- In Linux, each directory entry links a name to an inode number.
- The **inode** holds the file's metadata, including size, owner and permissions.

Inode 1925551:
Size: 6013654 bytes
File type: regular file
UID: 1001 (analyst)
GID: 1001 (analyst)
Also records permissions and access, modification and status-change timestamps.

```
analyst@ospc:~$ stat data/sales_2025_sample.csv
  File: data/sales_2025_sample.csv
  size: 6013654         Blocks: 11752      IO Block: 4096   regular file
Device: 0,51    Inode: 1925551     Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1001/ analyst)   Gid: ( 1001/ analyst)
Access: 2026-08-31 13:27:43.000000000 +0000
Modify: 2026-08-31 13:27:43.000000000 +0000
Change: 2026-08-31 13:28:19.309581990 +0000
 Birth: 2026-08-31 13:28:19.275582044 +0000
```

Directories

- **Directories** are a type of file that organizes names.
- In Linux, each directory entry links a name to an inode number.
- The inode holds metadata and information used to locate the file's data.
- The data are bytes stored in blocks. Their interpretation as characters depends on the encoding.

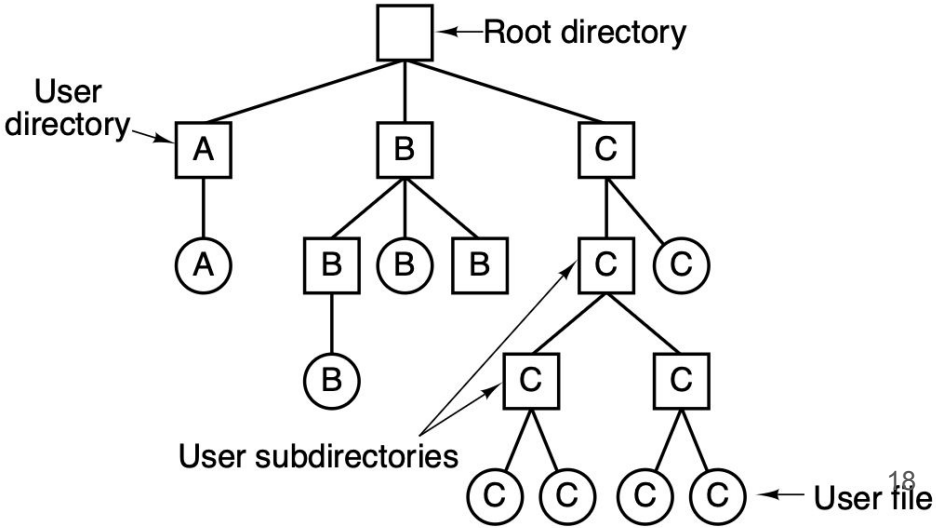


data blocks, 4 KB each
(this file: 1,469 of them)

```
analyst@ospc:~/data$ head -2 sales_2025_sample.csv | od -Ax -t x1z -v
000000 6f 72 64 65 72 5f 69 64 2c 6f 72 64 65 72 5f 64  >order_id,order_d<
000010 61 74 65 2c 72 65 67 69 6f 6e 2c 70 72 6f 64 75  >ate,region,produ<
000020 63 74 5f 63 61 74 65 67 6f 72 79 2c 70 72 6f 64  >ct_category,prod<
000030 75 63 74 5f 69 64 2c 71 75 61 6e 74 69 74 79 2c  >uct_id,quantity,<
000040 75 6e 69 74 5f 70 72 69 63 65 2c 63 75 73 74 6f  >unit_price,custo<
000050 6d 65 72 5f 69 64 2c 63 68 61 6e 6e 65 6c 0d 0a  >mer_id,channel..<
000060 31 2c 32 30 32 35 2d 31 31 2d 32 34 2c 49 62 65  >1,2025-11-24,Ibe<
000070 72 69 61 2c 53 70 6f 72 74 73 2c 50 35 35 30 36  >ria,Sports,P5506<
000080 2c 38 2c 35 32 2e 30 37 2c 43 32 33 34 33 34 2c  >,8,52.07,C23434,<
000090 73 74 6f 72 65 0d 0a                             >store..<
000097
```

Directory Hierarchy

- **Directories** group names, while subdirectories create a hierarchy.
- On Linux, the hierarchy starts at the root directory, `/`.
- Users normally have a home directory, e.g., `/home/analyst`. It is part of the hierarchy, not a separate system root.
- Permissions control who can access each directory and its contents.
- Modern systems commonly provide a directory hierarchy.
- Mounted file systems can join this hierarchy, even when they use different devices.
- Multics helped establish this design in the 1960s.



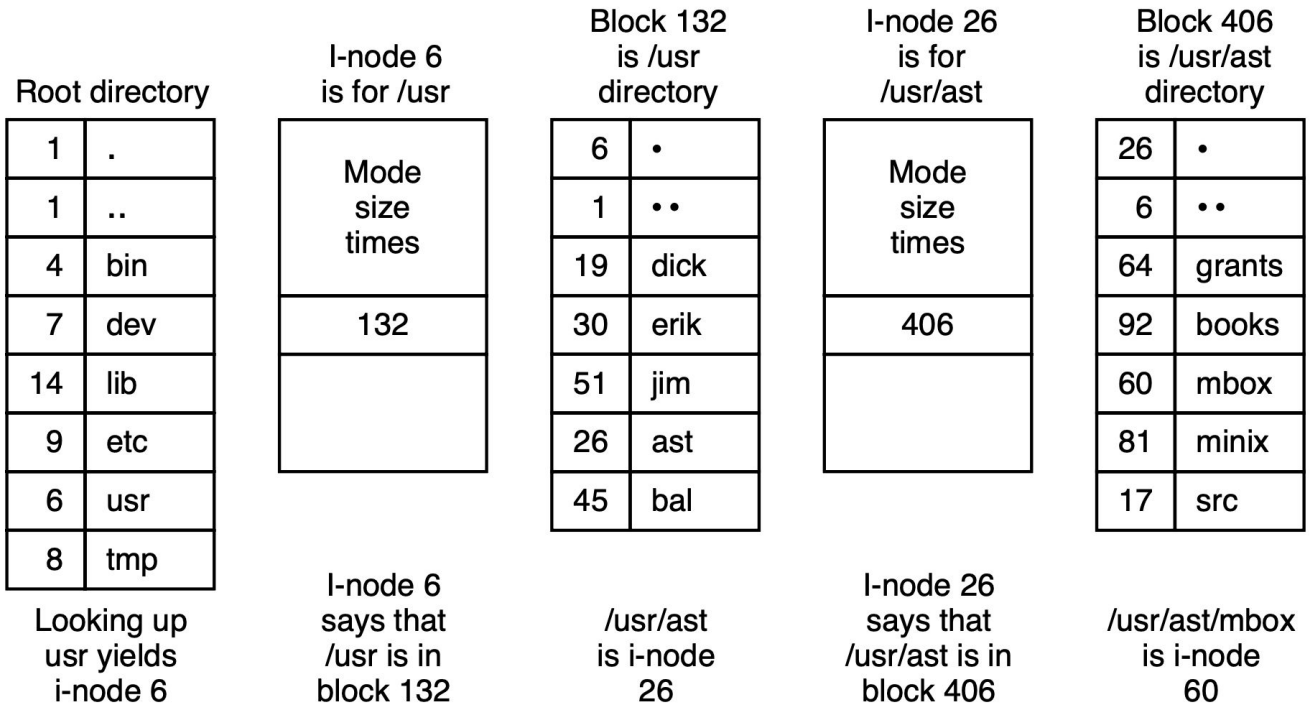
Curious Corner: Multics



- Modular hardware
- A single-level store
- Dynamic linking
- A command processor in user mode
- Directory hierarchies
- Access-control lists
- Photo: Honeywell 6180 (1973 model)

Directories

The steps in looking up */usr/ast/mbox*.



Path Names

- A path tells the OS where to look for a name in the directory hierarchy.
- An **absolute path** starts from a root:
  - Linux: `/home/analyst/data/sales_2025_sample.csv`
  - Windows: `C:\Users\analyst\data\sales.csv`
  - The same file can have several paths, e.g., through hard links.
- A **relative path** starts from the process's current working directory:
  - From `/home/analyst`, use `data/sales_2025_sample.csv`.
  - The shell command `pwd` shows that directory, while `cd` changes it.
- Question: does our container's shell use absolute or relative paths?

Path Names

- A path tells the OS where to look for a name in the directory hierarchy.
- An **absolute path** starts from a root:
  - Linux: `/home/analyst/data/sales_2025_sample.csv`
  - Windows: `C:\Users\analyst\data\sales.csv`
  - The same file can have several paths, e.g., through hard links.
- A **relative path** starts from the process's current working directory:
  - From `/home/analyst`, use `data/sales_2025_sample.csv`.
  - The shell command `pwd` shows that directory, while `cd` changes it.
- Both work. Choose a path that identifies the file from where the command runs.
- `.` refers to the current directory and `..` to its parent. At the root, `/..` still refers to `/`.

Directory Operations

- Directory operations are available through system calls and library functions.
- These examples apply to Linux in our container.

Interface	Description
`mkdir()`	Creates a directory. The names . and .. refer to that directory and its parent.
`rmdir()`	Removes an empty directory. The special names . and .. do not count as user entries.
`opendir()`	Opens a directory stream through the C library, allowing a program to discover its entries.
`closedir()`	Closes the directory stream and releases its resources.
`readdir()`	Returns the next entry through the C library, using OS support such as getdents().
`rename()`	Changes a directory's name or location within a file system, subject to permissions.

Directory Operations

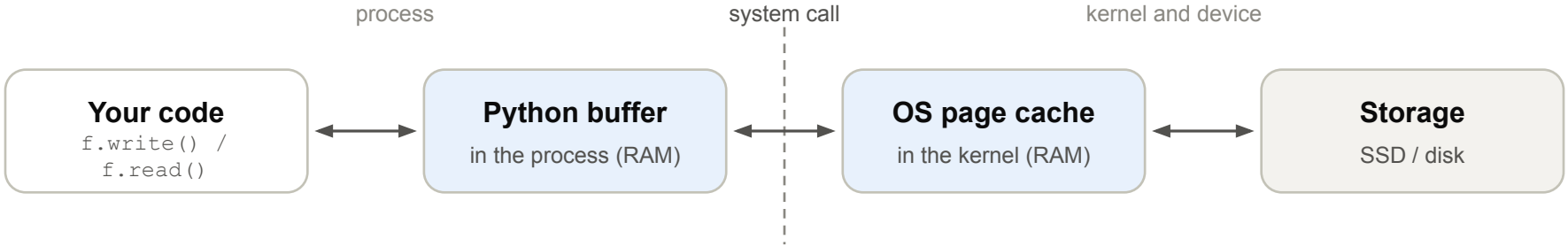
- Hard links and symbolic links connect names to files in different ways.

Interface	Description
`link()`	Creates another name for the same inode within one file system. Both names refer to the same data and metadata, so edits through either name affect the same file. A hard link is not a copy.
`unlink()`	Removes one name and reduces the hard-link count. Other names remain usable. The data can be reclaimed after the last hard link and the last open reference have gone.

- A **symbolic link** has its own inode and stores a target path. It can cross filesystem boundaries, but becomes dangling if that path no longer resolves.

Where Written Bytes Live

From write() to storage: what each layer saves, and what loses it



	Python buffer	OS page cache	Storage
Data moves on when	it fills, or on `f.flush()` or `close()`*	kernel writeback, usually within seconds, or `os.fsync()`	—
Lost if	the process dies (`kill -9`)	the power fails before writeback	neither: it survives both
A read here avoids	a system call	a storage access	—

* `close()` does not wait for the data to be copied to the storage device. Only `os.fsync()` does.